

Knowledge-Graph-Augmented Premise Retrieval for Neural Theorem Proving in Lean 4

Priyanka Agarwal

International Institute of Information Technology, Hyderabad
priyankaagarwal42875@gmail.com

Abstract

Neural theorem proving in Lean 4 is bottlenecked less by tactic generation than by *premise selection*: identifying, out of a library of hundreds of thousands of declarations, the handful actually relevant to a given proof state. We propose an architecture that augments a fixed-length tactic-sequence planner with a two-graph representation of the Mathlib library – a static premise-dependency graph and a dynamic per-proof-state graph, linked by signature and proof-dependency edges – retrieved over with a relation-aware graph neural network rather than flat dense retrieval, and wrapped in a lightweight stuck-recovery controller that reroutes search instead of exhausting a fixed budget on a dead branch. Unlike a purely speculative design, this proposal is grounded in a working small-scale reproduction: we cloned the reference implementation of the closest prior work (Petrovčič et al., 2025), patched it to run outside its original CUDA-only environment, and executed it end-to-end on real LeanDojo trace data from `lean4-example`, producing a verified, hand-checked ground-truth retrieval graph. We report what that trial run validated, what it left open, and a lower-risk AST-only fallback that remains fully valid if the graph-based track’s engineering cost outweighs its payoff.

1 Introduction

Interactive theorem provers such as Lean 4 make mathematical proof machine-checkable, but formalized mathematics remains a small, slowly-growing fraction of the informal mathematical literature: Mathlib’s several hundred thousand declarations are dwarfed by the tens of millions of theorem-like statements on the mathematics arXiv alone (Kurgan et al., 2026). Closing this gap with neural theorem provers

depends on two largely separate capabilities: (i) generating a plausible next tactic given the current proof state, and (ii) retrieving the right premises – lemmas, definitions, prior theorems – to cite in it. Retrieval is the harder and more studied problem: a semantically similar premise is not the same as a logically relevant one, and the premises a proof actually needs are frequently ones that share no surface vocabulary with the goal (Alemi et al., 2016).

This proposal targets premise retrieval specifically, via a knowledge-graph representation of Mathlib rather than the flat dense-retrieval baseline (ReProver) most current Lean provers use (Yang et al., 2023). The core claim is that library structure – which declaration’s signature references which other declaration, which premises a given tactic actually cites – carries retrieval signal that text similarity alone misses, and that this signal is best exposed as a graph and consumed with a graph neural network rather than folded into a single dense embedding.

What distinguishes this proposal from a purely architectural sketch is that its central component has already been built and run, not just designed. Section 4 reports a full, working reproduction of the closest published system on real data, including the specific engineering obstacles encountered and the gaps that remain – evidence a proposal built entirely on paper cannot offer.

2 Related Work

Classical premise selection. Symbolic relevance filtering predates neural methods by over a decade. MePo (Meng and Paulson, 2009) iteratively expands a relevant-fact set by symbol co-occurrence; SInE (Hoder and Voronkov, 2011) ranks axioms by inverse sym-

bol frequency and remains competitive on theories with millions of axioms, routinely finding proofs that need fewer than twenty. Isabelle’s Sledgehammer combines such filters with a post-hoc *minimization* step that shrinks an already-found premise set to a smaller sufficient one (Böhme and Nipkow, 2010), and later added the learned k-NN/naïve-Bayes ranker MaSh (Kühlwein et al., 2013). These systems establish the retrieval-then-reconstruct pattern this proposal follows, but operate over flat symbol features, not graph structure.

Neural and graph-based premise selection. DeepMath (Alemi et al., 2016) was among the first to apply sequence models to premise selection on the Mizar corpus. Wang et al. (Wang et al., 2017) moved to graph embeddings of formulae, and Paliwal et al. (Paliwal et al., 2020) established graph representations of higher-order-logic terms as a premise-selection feature source for HOList/HOL Light, the lineage this proposal’s graph encoder descends from. Magnushammer (Mikuła et al., 2023) later showed a contrastive transformer retriever alone can substantially outperform Sledgehammer (59.5% vs. 38.3% on PISA), establishing the ceiling neural retrieval can reach without any explicit graph structure – the baseline any graph-augmented method must beat to justify its added complexity.

Lean- and Coq-specific graph representations. Three recent systems are directly relevant. Petrovčič et al. (2025) build a heterogeneous graph over static *premise* nodes and dynamic per-tactic *state* nodes, connected by signature and proof-dependency edges, and train a relational graph convolutional network (RGCN) fused with text embeddings; on the LeanDojo Benchmark this beats the ReProver text-only baseline by over 25%. This is the system our proposed architecture extends and the one we reproduce in Section 4. Kurgan et al. (2026) take a complementary, larger-scale approach: LeanGraph extracts a six-edge-type declaration graph (`extends`, `field`, `sig`, `proof`, `def`, `docref`) directly from Lean’s kernel `Environment` API across Mathlib and 24 other projects (388K nodes, 11.3M edges), then bridges it to informal arXiv statements via embedded natural-language summaries.

Aniva et al. (2026) work at a different granularity entirely – ExprGraph represents individual Lean *expressions*, not library-scale dependencies, feeding a GNN that dispatches from a small, jointly complete set of atomic tactics. For Coq, Graph2Tac (Blaauwbroek et al., 2024) demonstrates *online* graph representation learning that updates as new definitions are seen, yielding a 1.5–1.72× improvement over offline baselines – a capability our static-graph design (Section 3) does not yet have and flags as future work.

Positioning. Graph-augmented premise retrieval for Lean is therefore no longer unclaimed territory (Petrovčič et al., 2025); this proposal’s contribution is (a) combining that retrieval mechanism with an explicit stuck-recovery controller and a unified multi-tactic closing model, neither of which appear in the cited systems, and (b) reporting a working, environment-adapted reproduction rather than a re-derivation of the same design from the paper alone.

3 Proposed Architecture

3.1 Representation

The proof state’s `Expr/Syntax` AST, extracted via Lean 4 metaprogramming, supplies local node features. Separately, two linked graphs are built from a pinned Mathlib commit using LeanDojo’s traced data: a *static premise graph* over definitions and lemmas, built once and reused across all training and retrieval, and a *dynamic state graph* with one node per proof state / tactic instance, generated per-theorem during data extraction and never baked into the static graph. The two are connected by *signature* edges (premises referenced in a state’s goal or hypotheses) and *proof-dependency* edges (premises a tactic actually used), both read directly from LeanDojo’s trace output rather than inferred by parsing tactic argument strings – the latter being the approach of an earlier iteration of this plan, which covered only the `rw` tactic and required regex-matching argument order.

3.2 Retrieval

Premise retrieval replaces ReProver-style dense nearest-neighbor search with a relation-aware GNN: an RGCN over the signature

and proof-dependency edges above, fused with text embeddings, following [Petrovčič et al. \(2025\)](#). The novelty this proposal can still claim on top of that published result is *multi-hop* traversal beyond a two-layer RGCN neighborhood – in particular structural **extends**/class-inheritance edges in the style of LeanGraph ([Kurgan et al., 2026](#)) – and reuse of the same graph for stuck-recovery rerouting, not only for the base retrieval step.

3.3 Planning and closing

Five independent models M_1 – M_5 , each predicting exactly k tactics ($k = 1, \dots, 5$), are wrapped in a ReProver-style best-first search over goal states. A single unified closing model, trained multi-class over `{rfl, simp, grind, omega, decide, not-yet-closable}`, replaces what would otherwise be five separate binary classifiers. Every predicted tactic is checked by the Lean compiler itself; there is no learned reward model.

3.4 Stuck-recovery controller

A lightweight monitor sits on top of the best-first search, tracking repeated goal states and no-progress steps. On trip, it forces a restart from the next-best branch, widens the graph-retrieval neighborhood, or falls back to the next M_k , rather than exhausting the search budget on a dead branch. This is deliberately simpler than the closest prior mechanism, a control-plane/data-plane cost-quality agent for Lean ([arXiv:2606.04883](#)), and is explicitly differentiated from it rather than presented as equivalent.

4 Preliminary Validation

To ground the retrieval component in more than a paper design, we cloned [Petrovčič et al.](#)’s reference implementation and ran it, unmodified in logic, end-to-end on real Lean-Dojo trace data from `lean4-example` and its full Lean 4 core dependency closure (907 files).

What we found. The released repository provides the *consumer* of a tagged premise-dependency corpus (a `Corpus` class, an RGCN, a `PremiseRetriever` built on a pretrained ByT5 encoder) but not the *producer* – the preprocessing step that tags each premise

with signature/proof provenance is not public. We wrote that step ourselves, using LeanDojo’s own `get_premise_definitions` and `get_annotated_tactic` APIs, and fed the result into their unmodified graph-construction code under their simplest supported tagging mode (`all_dojo`), which collapses the richer three-relation scheme into one generic **dependency** edge type.

Result. On a reduced corpus (4 files, 1,030 premises, 166 dependency edges) built from `lean4-example`’s two theorems and the Mathlib-core files defining the premises they cite, the pipeline correctly produced ground-truth retrieval labels: `hello_world`’s proof (`rw [add_assoc, add_comm b, ←add_assoc]`) was labeled with exactly `Nat.add_comm` and `Nat.add_assoc`, and `foo`’s proof (`rfl`, which cites no premise by name) correctly received zero labels. Those same two premises carried real dependency edges into other genuine Mathlib-core lemmas from their own proofs (e.g. `Nat.add_comm` $\rightarrow$ `Nat.add_left_comm`), confirming the static graph reflects actual library structure and not an artifact of the extraction script. A second run over the full 907-file, 19,239-premise closure was still executing after several hours on CPU-only hardware and was not required to validate correctness.

What this leaves open. Three gaps are disclosed rather than papered over: (1) the **signature** edge type – which premises appear in a state’s goal versus local context – is not yet reproduced, since its source preprocessing is unpublished; (2) the single generic **dependency** edge type is coarser than the paper’s own three-relation design; (3) only the two edges directly touching the example theorems were hand-verified against source, not the full edge set at scale. None of these affect the architecture proposed in Section 3, which is unchanged from the target design – they affect only how faithfully the current toy-scale trial reproduces it, and are listed here as the concrete first items for the implementation phase.

5 Alternative Path: AST-Only Fallback

The graph-based track carries measured, non-hypothetical engineering cost: an 8–10 hour CPU-only embedding pass for a single small repository’s dependency closure, undisclosed preprocessing that had to be approximated, and three platform-compatibility patches (a missing GPU-only `deepspeed` dependency, a C++ extension that failed to compile against the local macOS SDK, and a Python path resolution issue) needed just to run the reference implementation at all. If this cost stops being worth paying at any point in the project, the fallback is not a weaker design invented under pressure but Lean neural-proving’s already-validated baseline: the flat/tree-only AST as the sole representation, ReProver-style dense nearest-neighbor retrieval in place of the GNN, and the same M_1 – M_5 planners and unified closing model from Section 3, fed AST/state-text features directly. This path uses no graph libraries and no GNN training code, and – critically – is required regardless of which path is ultimately shipped, since it is the baseline the graph-augmented system must be benchmarked against to justify its added complexity at all.

6 Evaluation Plan

Training and held-out validation draw on LeanDojo Benchmark 4 (~122K theorems, 260K tactics) and LEAN-GitHub (~28.6K theorems, 219K tactics) as a supplementary training source. miniF2F (244 validation / 244 test) is used for both tuning and reporting; PutnamBench (1,709 problems) and ProofNet (~371 problems) are held out as test-only benchmarks the model never trains against. Retrieval quality is measured with Recall@k and reciprocal rank against ReProver’s dense-retrieval baseline, the same comparison Petrovčíč et al. (2025) report their >25% improvement against; end-to-end proving performance is measured as overall proof success rate under a fixed tactic-attempt budget, comparing the full graph-augmented system, the AST-only fallback of Section 5, and an ablation with the stuck-recovery controller disabled to isolate its individual contribution.

7 Conclusion

We propose extending a Lean 4 tactic-sequence planner with a knowledge-graph-augmented, GNN-based premise retriever, an explicit stuck-recovery controller, and a single unified closing model, and we ground the retrieval component in a working reproduction of the closest published system rather than a design sketch alone. The reproduction’s own results delimit the concrete next steps – recovering the missing signature-edge preprocessing, scaling past a toy repository, and reproducing the published benchmark numbers on real data – while the AST-only fallback ensures the project remains shippable even if the graph-based track’s cost proves prohibitive at scale.

References

- Alexander A. Alemi, François Chollet, Niklas Een, Geoffrey Irving, Christian Szegedy, and Josef Urban. 2016. DeepMath – deep sequence models for premise selection. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Leni Aniva, Iori Oikawa, David Dill, and Clark Barrett. 2026. Nazrin: Atomic tactics for graph neural networks for theorem proving in lean 4. *arXiv preprint arXiv:2602.18767*.
- Lasse Blaauwbroek, Miroslav Olšák, Jason Rute, Fidel Ivan Schaposnik Massolo, Jelle Piepenbrock, and Vasily Pestun. 2024. Graph2tac: On-line representation learning of formal math concepts. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- Sascha Böhme and Tobias Nipkow. 2010. Sledgehammer: Judgement day. In *International Joint Conference on Automated Reasoning (IJCAR)*.
- Kryštof Hoder and Andrei Voronkov. 2011. Sine qua non for large theory reasoning. In *International Conference on Automated Deduction (CADE)*.
- Daniel Kühlwein, Jasmin Christian Blanchette, Cezary Kaliszyk, and Josef Urban. 2013. MaSh: Machine learning for sledgehammer. In *International Conference on Interactive Theorem Proving (ITP)*.
- Simon Kurgan, Evan Wang, Eric Leonen, Sophie Szeto, Luke Alexander, Artemii Remizov, Jarod Alper, Giovanni Inchiostro, and Vasily Ilin. 2026. Theoremgraph: Bridging formal and informal mathematics. *arXiv preprint arXiv:2606.25363*.

- Jia Meng and Lawrence C. Paulson. 2009. Lightweight relevance filtering for machine-generated resolution problems. *Journal of Applied Logic*.
- Maciej Mikuła, Szymon Antoniak, Szymon Tworkowski, Albert Qiaochu Jiang, Jin Peng Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. 2023. Magnushammer: A transformer-based approach to premise selection. *arXiv preprint arXiv:2303.04488*.
- Aditya Paliwal, Sarah Loos, Markus Rabe, Kshitij Bansal, and Christian Szegedy. 2020. Graph representations for higher-order logic and theorem proving. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Job Petrovčič, David Eliecer Narvaez Denis, and Ljupčo Todorovski. 2025. Combining textual and structural information for premise selection in lean. *arXiv preprint arXiv:2510.23637*.
- Mingzhe Wang, Yihe Tang, Jian Wang, and Jia Deng. 2017. Premise selection for theorem proving by deep graph embedding. *arXiv preprint arXiv:1709.09994*.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*.